

APPLICATION FOR REGISTRATION OF COPYRIGHT

Literary Work - Computer Programme (Software)

TITLE OF THE WORK

SMART QUESTION PAPER GENERATOR - AUTOMATED QUESTION PAPER GENERATION SYSTEM WITH CYCLE-AWARE NON-REPETITION

CLASS OF WORK

Literary Work (Computer Programme / Software) - Section 2(o), Copyright Act, 1957

PROGRAMMING LANGUAGES

JavaScript (React), SQL / PL-pgSQL, CSS, HTML

NATURE OF APPLICANT'S INTEREST

Author and owner of the work

WHETHER PUBLISHED

_____ (Published / Unpublished)

YEAR OF FIRST PUBLICATION

YEAR OF CREATION

2026

COUNTRY OF FIRST PUBLICATION

India

AUTHOR / APPLICANT

PRADYUMNA GIRISH KULKARNI

VIMACO ENCLAVE, OPP. IG BUNGALOW, CAMP, AMRAVATI, MAHARASHTRA - 444602

Nationality: Indian

Co-authors: 9 joint authors. Particulars of all 10 authors are set out on the following page.

Date of this enclosure: 18 August 2026

PARTICULARS OF THE AUTHORS

The work is the joint work of the 10 authors named below. Entry 1 is the applicant; entries 2 to 10 are joint authors. All authors are citizens of India.

1. PRADYUMNA GIRISH KULKARNI

APPLICANT / AUTHOR

VIMACO ENCLAVE, OPP. IG BUNGALOW, CAMP, AMRAVATI, MAHARASHTRA - 444602

Nationality: Indian

2. SATCHIT ANIL DHAWALE

CO-AUTHOR

V.M.V ROAD, NEAR LILAI PHOTO STUDIO, KHARAYA NAGAR, MAHENDRA COLONY, VIDHARBH MAHAVIDYALA, AMRAVATI, MAHARASHTRA - 444604

Nationality: Indian

3. YASHWANT VIJAYRAO KIRAKTE

CO-AUTHOR

NANDURA BU, NANDURA BK, AMRAVATI, MAHARASHTRA - 444801

Nationality: Indian

4. AARYA SANJAY KUKADE

CO-AUTHOR

SWAROOP PLAZA APARTMENT, BEHIND CHAWLA BEKARY, ZADE LAY-OUT, RAMNAGAR, CHANDRAPUR, MAHARASHTRA - 442401

Nationality: Indian

5. VISHESH SANDEEPROAO PACHGHARE

CO-AUTHOR

WARD NO. 6, BHAVANI NAGAR KANDALI, PARATWADA, ACHALPUR, AMRAVATI, MAHARASHTRA - 444805

Nationality: Indian

6. AYUSH RUPESH DONGRE

CO-AUTHOR

BABULGAON, AKOLA, MAHARASHTRA - 444104

Nationality: Indian

7. NISHANT SUDHIR THAKARE

CO-AUTHOR

PLOT NO 10B, JAIGAURI NAGAR, NEAR ANUSAYAMATA MANDIR, DATTAWADI, AMRAVATI, MAHARASHTRA - 444603

Nationality: Indian

PARTICULARS OF THE AUTHORS (continued)

8. JANHAVI GOPAL HIRE

CO-AUTHOR

GANESHPUR, POGHAT, WASHIM, MAHARASHTRA - 444403

Nationality: Indian

9. VAISHNAVI RAMESH BAND

CO-AUTHOR

WARD NO. 01, TEACHERS COLONY PINJAR, AKOLA, MAHARASHTRA - 444407

Nationality: Indian

10. MAYUR S BHURANGE

CO-AUTHOR

FLAT NO. G002, VARAD RESIDENCY, GAJANAN NAGAR, BEHIND HIRWAL LAWN, KATHORA ROAD,
AMRAVATI, MAHARASHTRA - 444604

Nationality: Indian

DESCRIPTION OF THE WORK

Smart Question Paper Generator is a web-based software system that automates the preparation of examination question papers in an educational institution. The software maintains a unit-wise question bank, applies a reusable examination blueprint, selects questions under that blueprint, and produces a formatted question paper for download.

The distinguishing feature of the work is cycle-aware non-repetition. Every question issued in a generated paper is recorded against its examination cycle. When a further paper is generated in the same cycle - typically the backlog paper - the questions already issued are excluded at the database query level, so an overlap between the regular and the backlog paper cannot occur.

Question selection, non-repetition, sufficiency validation and audit logging are implemented as server-side database routines and triggers executing within a single transaction. If any requirement of the blueprint cannot be met, the operation is aborted in full and no partial paper is stored. The client application never performs the selection and never receives the pool of candidate questions.

PRINCIPAL MODULES OF THE SOFTWARE

1. Authentication and role-based access (administrator / teacher).
2. Question bank management - unit, marks, difficulty, Bloom's level, course outcome.
3. Paper format and blueprint management (reusable JSON blueprints).
4. Paper generation with cycle-aware non-repetition and sufficiency validation.
5. Document export - Word, PDF and image from a single representation.
6. Administration - subjects, teacher allotment, confirmed deletion.
7. Append-only audit log with timestamp, actor and action record.

TECHNOLOGY USED

React (Create React App) for the client; PostgreSQL for storage, with server-side routines, triggers and row-level security policies; JSON Web Token based authentication.

INDEX OF SOURCE FILES ENCLOSED

26 files - 3,254 lines of source code - 50 pages of listing. The SHA-256 digest of each file is given so that the enclosed listing can be verified against the submitted work.

FILE	LINES	SHA-256 (first 32 hex digits)
src/index.js	17	9b069b55a2f6884ddc0a86487e2b6447
src/App.js	78	5ee1005359523963dcb9a4fedc54f25a
src/supabaseClient.js	12	5cb4049b7d57fd5b2a8d9ea173cd5e18
src/lib/useProfile.js	31	5b1c306f2b3226d864cc2c760a78b700
src/lib/questionImages.js	75	2f61a7e8e649745b058f3e7b26212b6d
src/components/RequireRole.jsx	15	898d293432215ecda4d52d28c8fcc257
src/components/Nav.jsx	34	b1948d115943e2b54c2f7f4c3f476c55
src/components/DeleteSubjectDialog.jsx	94	982f68b8d8353e8bfb3edb21c448d5a2
src/pages/Login.jsx	116	601faa6a931b9bd9ae3ec0cffc29d1da8
src/pages/Home.jsx	47	c5a3df2466a1c50686e865343db839f6
src/pages/Dashboard.jsx	116	c59c9bce98e176d024e86b2d8c0d75f9
src/pages/TeacherHome.jsx	43	953e4605517d22ad2c74c2734bb94072
src/pages/QuestionBank.jsx	240	c68f6e16647c57e8d6c569b91b600bce
src/pages/GeneratePaper.jsx	134	b499cfacb0746b76110d479c9feafa64
src/pages/DownloadPaper.jsx	286	df8d2cc15479ed97457792d22a3a3672
src/pages/AdminHome.jsx	152	868601c28b473f321ff70e8bf92711e8
src/pages/AllPapers.jsx	124	f18fff34ab87fe61ef9cb44e4aae8227
src/pages/AuditLog.jsx	146	4bad5fadb20ef7dc7689335321bdfdf16
src/pages/Profile.jsx	97	09fb826ae5127c40115a7b808df0e1d2
src/pages/Settings.jsx	84	edf570372f870762da241bc410d1ecd3
src/App.css	352	dfcf574acb65d2112e661e1939c21270
src/pages.css	322	5071578b3a44b043aeaf217082b8ed72
public/index.html	51	a94d338ef0b5f66f82b5f5648a5e963d
admin_audit.sql	333	b9b2650a885c0878f889808df37c4621
question_images.sql	89	c92bfd843399f42fd8e7779af5d0cab6
seed_appmath2.sql	166	9065226473ea6707ac9f6ac35a86a294

DECLARATION

I declare that the source code enclosed with this application is the original work of the authors named in this enclosure; that it has been written by them and has not been copied from any other work; and that no portion of the enclosed listing has been blanked out, redacted or otherwise altered.

I further declare that the particulars stated in this enclosure are true to the best of my knowledge and belief, and that the complete source code of the work will be produced before the Registrar of Copyrights as and when required.

Place : _____

Date : _____

Signature of the applicant (for self and on behalf of the joint authors)

PRADYUMNA GIRISH KULKARNI

```
=====
FILE: src/index.js (17 lines)
=====
1 import React from 'react';
2 import ReactDOM from 'react-dom/client';
3 import { BrowserRouter } from 'react-router-dom';
4 import './index.css';
5 import App from './App';
6 import reportWebVitals from './reportWebVitals';
7
8 const root = ReactDOM.createRoot(document.getElementById('root'));
9 root.render(
10   <React.StrictMode>
11     <BrowserRouter>
12       <App />
13     </BrowserRouter>
14   </React.StrictMode>
15 );
16
17 reportWebVitals();

=====
FILE: src/App.js (78 lines)
=====
1 import { useState, useEffect } from 'react';
2 import { Routes, Route, Navigate } from 'react-router-dom';
3 import { supabase } from './supabaseClient';
4 import { useProfile } from './lib/useProfile';
5 import Login from './pages/Login';
6 import Nav from './components/Nav';
7 import RequireRole from './components/RequireRole';
8 import AdminHome from './pages/AdminHome';
9 import AllPapers from './pages/AllPapers';
10 import AuditLog from './pages/AuditLog';
11 import TeacherHome from './pages/TeacherHome';
12 import Home from './pages/Home';
13 import Dashboard from './pages/Dashboard';
14 import Profile from './pages/Profile';
15 import Settings from './pages/Settings';
16 import QuestionBank from './pages/QuestionBank';
17 import GeneratePaper from './pages/GeneratePaper';
18 import DownloadPaper from './pages/DownloadPaper';
19 import './App.css';
20 import './pages.css';
21
22 export default function App() {
23   const [session, setSession] = useState(null);
24   const [ready, setReady] = useState(false);
25
26   // Who is logged in?
27   useEffect(() => {
28     supabase.auth.getSession().then(({ data }) => {
29       setSession(data.session);
30       setReady(true);
31     });
32     const { data: sub } = supabase.auth.onAuthStateChange((_e, s) => setSession(s));
33     return () => sub.subscription.unsubscribe();
34   }, []);
35
36   if (!ready) return <div className="container"><p>Loading...</p></div>;
37   if (!session) return <Login />;
38
39   return (
40     <>
41       <Nav email={session.user.email} />
42       <div className="page">
43         <Routes>
44           <Route path="/" element={<Home />} />
```

```

45     <Route path="/dashboard" element={<Dashboard />} />
46     <Route path="/profile" element={<Profile />} />
47     <Route path="/settings" element={<Settings />} />
48     {/* Kept for links that expect role-based landing. */}
49     <Route path="/start" element={<HomeRedirect />} />
50     <Route
51       path="/admin"
52       element={<RequireRole role="admin"><AdminHome /></RequireRole>}
53     />
54     <Route
55       path="/admin/papers"
56       element={<RequireRole role="admin"><AllPapers /></RequireRole>}
57     />
58     <Route
59       path="/admin/audit"
60       element={<RequireRole role="admin"><AuditLog /></RequireRole>}
61     />
62     <Route path="/teacher" element={<TeacherHome />} />
63     <Route path="/subject/:id/bank" element={<QuestionBank />} />
64     <Route path="/subject/:id/generate" element={<GeneratePaper />} />
65     <Route path="/papers/:id/download" element={<DownloadPaper />} />
66     <Route path="*" element={<Navigate to="/" replace />} />
67   </Routes>
68 </div>
69 </>
70 );
71 }
72
73 // Send admins to /admin, teachers to /teacher.
74 function HomeRedirect() {
75   const { profile, loading } = useProfile();
76   if (loading) return <div className="card"><p>Loading...</p></div>;
77   return <Navigate to={profile?.role === 'admin' ? '/admin' : '/teacher'} replace />;
78 }

```

```
=====
FILE: src/supabaseClient.js   (12 lines)
=====
```

```

1 import { createClient } from '@supabase/supabase-js';
2
3 // Read config from .env.local (CRA requires the REACT_APP_ prefix).
4 // The anon/publishable key is PUBLIC by design - it ships in the browser
5 // bundle. Your data is protected by Row Level Security on the server, not by
6 // hiding this key. Never put the *service_role* key here.
7 const supabaseUrl = process.env.REACT_APP_SUPABASE_URL;
8 const supabaseAnonKey = process.env.REACT_APP_SUPABASE_ANON_KEY;
9
10 // One shared client for the whole app. Import THIS everywhere - never call
11 // createClient twice.
12 export const supabase = createClient(supabaseUrl, supabaseAnonKey);

```

```
=====
FILE: src/lib/useProfile.js   (31 lines)
=====
```

```

1 import { useState, useEffect } from 'react';
2 import { supabase } from '../supabaseClient';
3
4 // Fetch the current user's profile row (holds the role).
5 export function useProfile() {
6   const [profile, setProfile] = useState(null);
7   const [loading, setLoading] = useState(true);
8
9   useEffect(() => {
10     let alive = true;
11     (async () => {
12       const { data: { user } } = await supabase.auth.getUser();
13       if (!user) {
14         if (alive) setLoading(false);
15         return;

```

```

16     }
17     const { data } = await supabase
18       .from('profiles')
19       .select('id, full_name, role')
20       .eq('id', user.id)
21       .single();
22     if (alive) {
23       setProfile(data);
24       setLoading(false);
25     }
26   })();
27   return () => { alive = false; };
28 }, []);
29
30 return { profile, loading };
31 }

```

```

=====
FILE: src/lib/questionImages.js (75 lines)
=====
1 import { supabase } from '../supabaseClient';
2
3 export const BUCKET = 'question-images';
4 const ALLOWED = ['image/png', 'image/jpeg', 'image/webp', 'image/gif'];
5 const MAX_BYTES = 5 * 1024 * 1024;
6
7 // The object is named after the question, so a question row is enough to find
8 // its image. The extension is kept because it varies per upload.
9 export function objectPath(questionId, file) {
10   const ext = (file.name.split('.').pop() || 'png').toLowerCase();
11   return `${questionId}.${ext}`;
12 }
13
14 export function validateImage(file) {
15   if (!ALLOWED.includes(file.type)) return 'Use a PNG, JPG, WEBP or GIF image.';
16   if (file.size > MAX_BYTES) return 'Image must be 5 MB or smaller.';
17   return null;
18 }
19
20 // Upload and record the path on the question in one step. upsert:true so
21 // replacing an image reuses the same object name.
22 export async function attachImage(questionId, file) {
23   const problem = validateImage(file);
24   if (problem) throw new Error(problem);
25
26   const path = objectPath(questionId, file);
27   const { error: upErr } = await supabase.storage
28     .from(BUCKET)
29     .upload(path, file, { upsert: true, contentType: file.type });
30   if (upErr) throw upErr;
31
32   const { error } = await supabase
33     .from('questions')
34     .update({ image_url: path })
35     .eq('id', questionId);
36   if (error) throw error;
37   return path;
38 }
39
40 export async function removeImage(questionId, path) {
41   if (path) await supabase.storage.from(BUCKET).remove([path]);
42   const { error } = await supabase
43     .from('questions')
44     .update({ image_url: null })
45     .eq('id', questionId);
46   if (error) throw error;
47 }
48
49 // Batch-sign a set of object paths. Private bucket, so every read needs one.

```

```

50 export async function signedUrls(paths, expiresIn = 3600) {
51   const list = [...new Set(paths.filter(Boolean))];
52   if (list.length === 0) return {};
53   const { data, error } = await supabase.storage
54     .from(BUCKET)
55     .createSignedUrls(list, expiresIn);
56   if (error) throw error;
57   return Object.fromEntries(
58     (data || []).filter((d) => d.signedUrl).map((d) => [d.path, d.signedUrl])
59   );
60 }
61
62 // Signed URLs expire and are cross-origin. Inlining as a data URL makes the
63 // image survive in the word file and lets html2canvas paint it without a
64 // CORS round-trip at capture time.
65 export async function toDataURL(url) {
66   const res = await fetch(url);
67   if (!res.ok) throw new Error(`Could not load image (${res.status})`);
68   const blob = await res.blob();
69   return await new Promise((resolve, reject) => {
70     const fr = new FileReader();
71     fr.onload = () => resolve(fr.result);
72     fr.onerror = () => reject(new Error('Could not read image data.'));
73     fr.readAsDataURL(blob);
74   });
75 }

```

```

=====
FILE: src/components/RequireRole.jsx (15 lines)
=====

```

```

1 import { useProfile } from '../lib/useProfile';
2
3 // Gate a route behind a role. Renders children only for the matching role.
4 export default function RequireRole({ role, children }) {
5   const { profile, loading } = useProfile();
6   if (loading) return <div className="card"><p>Loading...</p></div>;
7   if (profile?.role !== role) {
8     return (
9       <div className="card">
10         <p className="msg">Access denied - requires {role} role.</p>
11       </div>
12     );
13   }
14   return children;
15 }

```

```

=====
FILE: src/components/Nav.jsx (34 lines)
=====

```

```

1 import { Link, NavLink } from 'react-router-dom';
2 import { supabase } from '../supabaseClient';
3 import { useProfile } from '../lib/useProfile';
4
5 export default function Nav({ email }) {
6   const { profile } = useProfile();
7   const admin = profile?.role === 'admin';
8
9   return (
10     <nav className="nav">
11       <Link to="/" className="brand">
12         <img
13           className="brand-mark"
14           src={` ${process.env.PUBLIC_URL}/crest.jpg` }
15           alt=""
16           width="28"
17           height="28"
18         />
19       <span>Paper Generator</span>
20     </Link>

```

```

21     <NavLink to="/dashboard">Dashboard</NavLink>
22     {admin
23       ? <NavLink to="/admin">Admin</NavLink>
24       : <NavLink to="/teacher">My subjects</NavLink>}
25     <NavLink to="/profile">Profile</NavLink>
26     <NavLink to="/settings">Settings</NavLink>
27     <span className="nav-spacer" />
28     <span className="nav-email">{email}</span>
29     <button className="btn-sm" onClick={() => supabase.auth.signOut()}>
30       Sign out
31     </button>
32   </nav>
33 );
34 }

```

=====

FILE: src/components/DeleteSubjectDialog.jsx (94 lines)

=====

```

1  import { useEffect, useState } from 'react';
2  import { supabase } from '../supabaseClient';
3
4  // Confirmation dialog for deleting a subject.
5  //
6  // The typed-code check here is a courtesy, not the safeguard - delete_subject()
7  // re-checks the code and the admin role server-side, so calling the RPC
8  // directly from the API without them fails exactly the same way.
9  export default function DeleteSubjectDialog({ subject, onClose, onDelete }) {
10   const [preview, setPreview] = useState(null);
11   const [typed, setTyped] = useState('');
12   const [busy, setBusy] = useState(false);
13   const [err, setErr] = useState('');
14
15   // Ask the database what this delete would destroy, counted live.
16   useEffect(() => {
17     (async () => {
18       const { data, error } = await supabase.rpc('subject_delete_preview', {
19         p_subject_id: subject.id,
20       });
21       if (error) setErr(error.message);
22       else setPreview(data);
23     })();
24   }, [subject.id]);
25
26   const armed = typed.trim() === subject.code;
27
28   async function confirmDelete() {
29     setBusy(true);
30     setErr('');
31     const { data, error } = await supabase.rpc('delete_subject', {
32       p_subject_id: subject.id,
33       p_confirm_code: typed.trim(),
34     });
35     setBusy(false);
36     if (error) return setErr(error.message);
37     onDelete(subject, data);
38   }
39
40   const n = (k) => Number(preview?.[k] ?? 0);
41
42   return (
43     <div className="modal-back" onClick={onClose}>
44       <div className="modal" onClick={(e) => e.stopPropagation()}>
45         <h3 style={{ marginTop: 0 }}>Delete {subject.code}</h3>
46         <p className="subtitle">{subject.name}</p>
47
48         {err && <p className="msg">{err}</p>}
49         {!preview && !err && <p className="subtitle">Counting what will be lost...</p>}
50
51         {preview && (

```

```

52     <>
53     <p className="warn-box">
54       This cannot be undone. Deleting this subject also permanently removes:
55     </p>
56     <ul className="impact">
57       <li><b>{n('questions')}</b> questions from the bank</li>
58       <li><b>{n('papers')}</b> generated papers (and their contents)</li>
59       <li><b>{n('cycles')}</b> exam cycles</li>
60       <li><b>{n('allotments')}</b> teacher allotments</li>
61       <li><b>{n('formats')}</b> subject-specific paper formats</li>
62     </ul>
63     <p className="subtitle">
64       Shared formats (used by other subjects) are not touched. The deletion
65       is recorded in the audit log with these counts.
66     </p>
67
68     <label>Type <b>{subject.code}</b> to confirm</label>
69     <input
70       autoFocus
71       value={typed}
72       onChange={(e) => setTyped(e.target.value)}
73       placeholder={subject.code}
74     />
75
76     <div className="row" style={{ justifyContent: 'flex-end' }}>
77       <button type="button" className="btn-sm" onClick={onClose} disabled={busy}>
78         Cancel
79       </button>
80       <button
81         type="button"
82         className="submit compact danger-btn"
83         disabled={!armed || busy}
84         onClick={confirmDelete}
85       >
86         {busy ? 'Deleting...' : 'Delete permanently'}
87       </button>
88     </div>
89   </>
90   )}
91 </div>
92 </div>
93 );
94 }

```

```

=====
FILE: src/pages/Login.jsx (116 lines)
=====

```

```

1 import { useState } from 'react';
2 import { supabase } from '../supabaseClient';
3
4 export default function Login() {
5   const [email, setEmail] = useState('');
6   const [password, setPassword] = useState('');
7   const [fullName, setFullName] = useState('');
8   const [isSignIn, setIsSignIn] = useState(true);
9   const [loading, setLoading] = useState(false);
10  const [message, setMessage] = useState('');
11
12  async function handleSignUp() {
13    setLoading(true);
14    setMessage('');
15    const { data, error } = await supabase.auth.signUp({
16      email,
17      password,
18      options: { data: { full_name: fullName } },
19    });
20    if (error) setMessage(error.message);
21    else if (!data.session) setMessage('Check your email to confirm, then sign in.');
```

```
23     setLoading(false);
24   }
25
26   async function handleSignIn() {
27     setLoading(true);
28     setMessage('');
29     const { error } = await supabase.auth.signInWithPassword({ email, password });
30     if (error) setMessage(error.message);
31     setLoading(false);
32   }
33
34   function handleSubmit(e) {
35     e.preventDefault();
36     if (isSignIn) handleSignIn();
37     else handleSignUp();
38   }
39
40   return (
41     <div className="container">
42       <div className="card">
43         {/* PUBLIC_URL, not a relative path - Login renders at whatever route the
44           signed-out user landed on, including nested ones like /subject/x/bank. */}
45         <img
46           className="crest"
47           src={`${process.env.PUBLIC_URL}/crest.jpg`}
48           alt="P. R. Pote Patil College of Engineering & Management, Amravati"
49           width="96"
50           height="96"
51         />
52         <p className="eyebrow">Department of Computer Science & Engineering</p>
53         <h1>Smart Question Paper Generator</h1>
54         <p className="subtitle">
55           {isSignIn
56             ? 'Sign in to reach your subjects and question bank.'
57             : 'Create an account. An admin will allot your subjects.'}
58         </p>
59
60         <form onSubmit={handleSubmit}>
61           {!isSignIn && (
62             <>
63               <label htmlFor="fullName">Full name</label>
64               <input
65                 id="fullName"
66                 type="text"
67                 value={fullName}
68                 onChange={(e) => setFullName(e.target.value)}
69                 required
70               />
71             </>
72           )}
73
74           <label htmlFor="email">Email</label>
75           <input
76             id="email"
77             type="email"
78             autoComplete="email"
79             value={email}
80             onChange={(e) => setEmail(e.target.value)}
81             required
82           />
83
84           <label htmlFor="password">Password</label>
85           <input
86             id="password"
87             type="password"
88             autoComplete={isSignIn ? 'current-password' : 'new-password'}
89             value={password}
90             onChange={(e) => setPassword(e.target.value)}
91             required
```

```

92         />
93
94         <button className="submit" type="submit" disabled={loading}>
95             {loading ? 'Please wait...' : isSignIn ? 'Sign in' : 'Create account'}
96         </button>
97     </form>
98
99     <p className="toggle">
100         {isSignIn ? "Don't have an account? " : 'Already have an account? '}
101         <button
102             type="button"
103             onClick={() => {
104                 setIsSignIn(!isSignIn);
105                 setMessage('');
106             }}
107         >
108             {isSignIn ? 'Sign up' : 'Sign in'}
109         </button>
110     </p>
111
112     {message && <p className="msg">{message}</p>}
113 </div>
114 </div>
115 );
116 }

```

```

=====
FILE: src/pages/Home.jsx (47 lines)
=====

```

```

1 import { Link } from 'react-router-dom';
2 import { useProfile } from '../lib/useProfile';
3
4 // Landing page. Role-aware list of entry points - no data fetching of its own.
5 export default function Home() {
6     const { profile, loading } = useProfile();
7     if (loading) return <div className="card"><p>Loading...</p></div>;
8
9     const admin = profile?.role === 'admin';
10
11     return (
12         <div className="card wide">
13             <h1>Smart Question Paper Generator</h1>
14             <p className="subtitle">
15                 Signed in as {profile?.full_name || 'user'} ({profile?.role || 'teacher'})
16             </p>
17
18             <h3>Go to</h3>
19             <ul>
20                 <li><Link to="/dashboard">Dashboard</Link> - counts and recent activity</li>
21                 {admin ? (
22                     <>
23                         <li><Link to="/admin">Administrator Control Panel</Link> - subjects, allotments, d
24                         <li><Link to="/admin/papers">All generated papers</Link></li>
25                         <li><Link to="/admin/audit">Audit log viewer</Link></li>
26                     </>
27                 ) : (
28                     <li><Link to="/teacher">My subjects</Link> - question bank and paper generation</li>
29                 )}
30                 <li><Link to="/profile">User profile</Link></li>
31                 <li><Link to="/settings">Settings</Link></li>
32             </ul>
33
34             <h3>How a paper is produced</h3>
35             <ol>
36                 <li>Maintain the unit-wise question bank for a subject.</li>
37                 <li>Pick a paper format, an exam cycle and the variant (normal or backlog).</li>
38                 <li>The database selects the questions and stores the paper.</li>
39                 <li>Download it as Word, PDF or image.</li>

```

```

40     </ol>
41     <p className="subtitle">
42         Questions issued in an exam cycle are excluded from later papers in the same
43         cycle, so the backlog paper can never repeat the normal paper.
44     </p>
45 </div>
46 );
47 }

=====
FILE: src/pages/Dashboard.jsx (116 lines)
=====
1  import { useEffect, useState, useCallback } from 'react';
2  import { Link } from 'react-router-dom';
3  import { supabase } from '../supabaseClient';
4  import { useProfile } from '../lib/useProfile';
5
6  const fmt = (s) =>
7      s ? new Date(s).toLocaleString('en-IN', { dateStyle: 'medium', timeStyle: 'short' })
8      : '-';
9
10 // Role-aware summary. Admin sees institute-wide totals, teacher sees their own.
11 // All counts use head:true so no rows are transferred, only the count.
12 export default function Dashboard() {
13     const { profile, loading: pLoading } = useProfile();
14     const [stats, setStats] = useState(null);
15     const [recent, setRecent] = useState([]);
16     const [err, setErr] = useState('');
17
18     const count = useCallback(async (table, filter) => {
19         let q = supabase.from(table).select('*', { count: 'exact', head: true });
20         if (filter) q = filter(q);
21         const { count: n, error } = await q;
22         if (error) throw error;
23         return n ?? 0;
24     }, []);
25
26     useEffect(() => {
27         if (pLoading) return;
28         (async () => {
29             try {
30                 const admin = profile?.role === 'admin';
31                 const next = {};
32                 if (admin) {
33                     next.subjects = await count('subjects');
34                     next.questions = await count('questions');
35                     next.papers = await count('papers');
36                     next.audit = await count('audit_logs');
37                 } else {
38                     // RLS already scopes these to the signed-in teacher.
39                     next.subjects = await count('teacher_subjects');
40                     next.questions = await count('questions');
41                     next.papers = await count('papers');
42                 }
43                 setStats(next);
44
45                 const { data } = await supabase
46                     .from('papers')
47                     .select('id, title, variant, created_at, subjects(code)')
48                     .order('created_at', { ascending: false })
49                     .limit(5);
50                 setRecent(data || []);
51             } catch (e) {
52                 setErr(e.message);
53             }
54         })();
55     }, [pLoading, profile, count]);
56
57     if (pLoading || (!stats && !err)) return <div className="card"><p>Loading...</p></div>;

```

```

58
59   const admin = profile?.role === 'admin';
60
61   return (
62     <div className="card wide">
63       <h1>Dashboard</h1>
64       <p className="subtitle">
65         {admin ? 'Institute-wide totals' : 'Your subjects and papers'}
66       </p>
67       {err && <p className="msg">{err}</p>}
68
69       <div className="subj-grid">
70         <div className="subj-card">
71           <b>{stats?.subjects ?? 0}</b>
72           <span>{admin ? 'Subjects' : 'Subjects allotted'}</span>
73         </div>
74         <div className="subj-card">
75           <b>{stats?.questions ?? 0}</b>
76           <span>Questions in bank</span>
77         </div>
78         <div className="subj-card">
79           <b>{stats?.papers ?? 0}</b>
80           <span>Papers generated</span>
81         </div>
82         {admin && (
83           <div className="subj-card">
84             <b>{stats?.audit ?? 0}</b>
85             <span>Audit entries</span>
86           </div>
87         )}
88       </div>
89
90       <h3>Recent papers</h3>
91       {recent.length === 0 && <p className="subtitle">None yet.</p>}
92       {recent.map((p) => (
93         <div key={p.id} className="q-row">
94           <span className="q-meta">{fmt(p.created_at)}</span>
95           <span className="q-text">
96             {p.subjects?.code || '-'} - {p.title} ({p.variant})
97           </span>
98           <Link className="btn-sm" to={`papers/${p.id}/download`}>Download</Link>
99         </div>
100       ))}
101
102       <h3>Shortcuts</h3>
103       <p className="subtitle">
104         {admin ? (
105           <>
106             <Link to="/admin">Control panel</Link> -{' '}
107             <Link to="/admin/papers">All papers</Link> -{' '}
108             <Link to="/admin/audit">Audit log</Link>
109           </>
110         ) : (
111           <Link to="/teacher">My subjects</Link>
112         )}
113       </p>
114     </div>
115   );
116 }

```

```

=====
FILE: src/pages/TeacherHome.jsx (43 lines)
=====

```

```

1 import { useEffect, useState } from 'react';
2 import { Link } from 'react-router-dom';
3 import { supabase } from '../supabaseClient';
4
5 // Teacher landing: subjects allotted to me by admin.
6 export default function TeacherHome() {

```

PAGES OMITTED

The complete source code listing of this work runs to 50 pages. In accordance with the practice of the Copyright Office for computer programmes, only the first 10 pages and the last 10 pages of the listing are enclosed.

Source code pages 11 to 40 are omitted here. No portion of the enclosed pages has been blanked out, redacted or altered. The complete source code will be produced before the Registrar of Copyrights on demand.

```

44     You can add webfonts, meta tags, or analytics to this file.
45     The build step will place the bundled scripts into the <body> tag.
46
47     To begin the development, run `npm start` or `yarn start`.
48     To create a production bundle, use `npm run build` or `yarn build`.
49     -->
50 </body>
51 </html>

=====
FILE: admin_audit.sql (333 lines)
=====
1  -- =====
2  -- TeacherEase - admin features: audit log, subject deletion,
3  -- admin-wide visibility of generated papers.
4  --
5  -- Run in the Supabase SQL editor of project dfkpsipqdkbrknxmtzmh.
6  -- Idempotent: safe to run more than once.
7  --
8  -- Design notes
9  -- * audit_logs holds NO foreign keys to the entities it records.
10 --   A FK to subjects would cascade-delete the very row that proves
11 --   the subject was deleted. Identity is kept as a plain uuid plus a
12 --   denormalised snapshot in `details`.
13 -- * Clients can never INSERT into audit_logs. Rows are written only by
14 --   SECURITY DEFINER triggers and functions, so the log cannot be
15 --   forged or skipped by calling the REST API directly.
16 -- * audit_logs has no UPDATE or DELETE policy for anyone. Append-only.
17 -- =====
18
19 begin;
20
21 -- -----
22 -- 1. Role helper
23 -- SECURITY DEFINER so it bypasses RLS on profiles - otherwise a
24 -- policy on profiles that calls is_admin() would recurse.
25 -- -----
26 create or replace function public.is_admin()
27 returns boolean
28 language sql
29 stable
30 security definer
31 set search_path = public
32 as $$
33     select exists (
34         select 1 from public.profiles
35         where id = auth.uid() and role = 'admin'
36     );
37 $$;
38
39 -- -----
40 -- 2. Audit log table
41 -- -----
42 create table if not exists public.audit_logs (
43     id          bigserial primary key,
44     at          timestampz not null default now(), -- when
45     actor_id    uuid, -- who (null = SQL editor / system)
46     actor_email text,
47     actor_role  text,
48     action      text not null, -- 'subject.delete', 'paper.generate', ...
49     entity      text not null, -- 'subject' | 'paper' | 'question' | 'allotment'
50     entity_id   uuid, -- deliberately NOT a foreign key
51     summary     text, -- one human-readable line
52     details     jsonb -- snapshot + counts, survives deletion
53 );
54
55 create index if not exists audit_logs_at_idx on public.audit_logs (at desc);
56 create index if not exists audit_logs_action_idx on public.audit_logs (action);
57 create index if not exists audit_logs_entity_idx on public.audit_logs (entity, entity_id);

```

```
58
59 alter table public.audit_logs enable row level security;
60
61 -- Admins read. Nobody writes, updates or deletes through the API:
62 -- the absence of those policies is the enforcement.
63 drop policy if exists audit_logs_admin_select on public.audit_logs;
64 create policy audit_logs_admin_select on public.audit_logs
65     for select using (public.is_admin());
66
67 -- -----
68 -- 3. Writer used by every trigger and RPC below
69 -- -----
70 create or replace function public.log_audit(
71     p_action text,
72     p_entity text,
73     p_id      uuid,
74     p_summary text,
75     p_details jsonb default '{} '::jsonb
76 ) returns void
77 language plpgsql
78 security definer
79 set search_path = public
80 as $$
81 declare
82     v_email text;
83     v_role  text;
84 begin
85     select u.email into v_email from auth.users u where u.id = auth.uid();
86     select p.role   into v_role   from public.profiles p where p.id = auth.uid();
87
88     insert into public.audit_logs
89         (actor_id, actor_email, actor_role, action, entity, entity_id, summary, details)
90     values
91         (auth.uid(), v_email, v_role, p_action, p_entity, p_id, p_summary, p_details);
92 end;
93 $$;
94
95 -- -----
96 -- 4. Automatic logging via triggers
97 --     Triggers, not client-side calls: the log cannot be bypassed by a
98 --     client that "forgets" to write it, and paper generation gets logged
99 --     without touching generate_paper().
100 -- -----
101
102 -- subjects: created
103 create or replace function public.audit_subject_ins()
104 returns trigger language plpgsql security definer set search_path = public as $$
105 begin
106     perform public.log_audit(
107         'subject.create', 'subject', new.id,
108         format('Created subject %s - %s', new.code, new.name),
109         jsonb_build_object('code', new.code, 'name', new.name, 'semester', new.semester));
110     return new;
111 end $$;
112
113 drop trigger if exists trg_audit_subject_ins on public.subjects;
114 create trigger trg_audit_subject_ins
115     after insert on public.subjects
116     for each row execute function public.audit_subject_ins();
117
118 -- papers: generated
119 create or replace function public.audit_paper_ins()
120 returns trigger language plpgsql security definer set search_path = public as $$
121 declare v_code text; v_cycle text;
122 begin
123     select code into v_code from public.subjects where id = new.subject_id;
124     select name into v_cycle from public.exam_cycles where id = new.cycle_id;
125     perform public.log_audit(
126         'paper.generate', 'paper', new.id,
```

```
127     format('Generated %s paper "%s" for %s (%s)',
128           coalesce(new.variant, '?'), coalesce(new.title, 'untitled'),
129           coalesce(v_code, '?'), coalesce(v_cycle, 'no cycle')),
130     jsonb_build_object('subject_code', v_code, 'cycle', v_cycle,
131                       'variant', new.variant, 'total_marks', new.total_marks));
132     return new;
133 end $$;
134
135 drop trigger if exists trg_audit_paper_ins on public.papers;
136 create trigger trg_audit_paper_ins
137   after insert on public.papers
138   for each row execute function public.audit_paper_ins();
139
140 -- questions: hard-deleted from the bank
141 create or replace function public.audit_question_del()
142 returns trigger language plpgsql security definer set search_path = public as $$
143 begin
144     -- Skip rows removed as part of a subject deletion; that RPC logs one
145     -- summary row instead of hundreds of individual ones.
146     if current_setting('teacherease.bulk_delete', true) = 'on' then
147         return old;
148     end if;
149     perform public.log_audit(
150         'question.delete', 'question', old.id,
151         format('Deleted question (Unit %s, %s marks)', old.unit_no, old.marks),
152         jsonb_build_object('unit_no', old.unit_no, 'marks', old.marks,
153                           'text', old.text, 'subject_id', old.subject_id));
154     return old;
155 end $$;
156
157 drop trigger if exists trg_audit_question_del on public.questions;
158 create trigger trg_audit_question_del
159   before delete on public.questions
160   for each row execute function public.audit_question_del();
161
162 -- teacher_subjects: allotment granted / revoked
163 create or replace function public.audit_allotment()
164 returns trigger language plpgsql security definer set search_path = public as $$
165 declare v_code text; v_teacher text; v_row record;
166 begin
167     v_row := coalesce(new, old);
168     select code      into v_code      from public.subjects where id = v_row.subject_id;
169     select full_name into v_teacher from public.profiles where id = v_row.teacher_id;
170
171     if tg_op = 'INSERT' then
172         perform public.log_audit('allotment.grant', 'allotment', v_row.subject_id,
173             format('Alloted %s to %s', coalesce(v_code, '?'), coalesce(v_teacher, 'teacher')),
174             jsonb_build_object('subject_code', v_code, 'teacher_id', v_row.teacher_id));
175         return new;
176     else
177         if current_setting('teacherease.bulk_delete', true) = 'on' then
178             return old;
179         end if;
180         perform public.log_audit('allotment.revoke', 'allotment', v_row.subject_id,
181             format('Revoked %s from %s', coalesce(v_code, '?'), coalesce(v_teacher, 'teacher')),
182             jsonb_build_object('subject_code', v_code, 'teacher_id', v_row.teacher_id));
183         return old;
184     end if;
185 end $$;
186
187 drop trigger if exists trg_audit_allotment_ins on public.teacher_subjects;
188 create trigger trg_audit_allotment_ins
189   after insert on public.teacher_subjects
190   for each row execute function public.audit_allotment();
191
192 drop trigger if exists trg_audit_allotment_del on public.teacher_subjects;
193 create trigger trg_audit_allotment_del
194   before delete on public.teacher_subjects
195   for each row execute function public.audit_allotment();
```

```

196
197 -- -----
198 -- 5. Delete preview - what the admin is about to destroy.
199 --     Read-only. Feeds the confirmation dialog.
200 -- -----
201 create or replace function public.subject_delete_preview(p_subject_id uuid)
202 returns jsonb
203 language plpgsql
204 stable
205 security definer
206 set search_path = public
207 as $$
208 declare v jsonb; v_code text; v_name text;
209 begin
210     if not public.is_admin() then
211         raise exception 'Only an admin can delete a subject.';
212     end if;
213
214     select code, name into v_code, v_name from public.subjects where id = p_subject_id;
215     if v_code is null then
216         raise exception 'Subject not found.';
217     end if;
218
219     select jsonb_build_object(
220         'code', v_code,
221         'name', v_name,
222         'questions', (select count(*) from public.questions      where subject_id = p_subject_id)
223         >> ,
224         'papers',    (select count(*) from public.papers        where subject_id = p_subject_id)
225         >> ,
226         'cycles',    (select count(*) from public.exam_cycles    where subject_id = p_subject_id)
227         >> ,
228         'allotments', (select count(*) from public.teacher_subjects where subject_id = p_subject_id)
229         >> d),
230         'formats',    (select count(*) from public.paper_formats  where subject_id = p_subject_id)
231     ) into v;
232     return v;
233 end $$;
234
235 -- -----
236 -- 6. Delete subject
237 --     * admin only, checked server-side
238 --     * confirmation checked server-side too: the caller must echo the
239 --       subject code back exactly. Calling the RPC straight from the API
240 --       without the code fails the same way the UI would.
241 --     * whole thing is one transaction - it all goes or nothing goes
242 --     * counts are captured BEFORE deletion and stored in the audit row,
243 --       because afterwards there is nothing left to count
244 -- -----
245 create or replace function public.delete_subject(
246     p_subject_id uuid,
247     p_confirm_code text
248 ) returns jsonb
249 language plpgsql
250 security definer
251 set search_path = public
252 as $$
253 declare
254     v_code    text;
255     v_name    text;
256     v_counts  jsonb;
257 begin
258     if not public.is_admin() then
259         raise exception 'Only an admin can delete a subject.';
260     end if;
261
262     select code, name into v_code, v_name
263     from public.subjects where id = p_subject_id;
264

```

```

261     if v_code is null then
262         raise exception 'Subject not found (already deleted?).';
263     end if;
264
265     if btrim(coalesce(p_confirm_code, '')) is distinct from v_code then
266         raise exception 'Confirmation failed - type the subject code % exactly to delete it.', v_c
            >> ode;
267     end if;
268
269     -- Snapshot before anything is destroyed.
270     v_counts := public.subject_delete_preview(p_subject_id);
271
272     -- Silence the per-row question/allotment audit triggers for this cascade;
273     -- one summary row is written below instead.
274     perform set_config('teacherease.bulk_delete', 'on', true); -- true = transaction-local
275
276     -- Children first, deepest level out.
277     delete from public.paper_questions
278         where paper_id in (select id from public.papers where subject_id = p_subject_id);
279     delete from public.papers         where subject_id = p_subject_id;
280     delete from public.exam_cycles     where subject_id = p_subject_id;
281     delete from public.questions      where subject_id = p_subject_id;
282     delete from public.teacher_subjects where subject_id = p_subject_id;
283     -- Only subject-specific formats. subject_id IS NULL means a shared/global
284     -- format used by other subjects - never touch those.
285     delete from public.paper_formats   where subject_id = p_subject_id;
286     delete from public.subjects        where id = p_subject_id;
287
288     perform set_config('teacherease.bulk_delete', 'off', true);
289
290     perform public.log_audit(
291         'subject.delete', 'subject', p_subject_id,
292         format('DELETED subject %s - %s (%s questions, %s papers, %s cycles)',
293             v_code, v_name,
294             v_counts->>'questions', v_counts->>'papers', v_counts->>'cycles'),
295         v_counts || jsonb_build_object('confirmed_with', p_confirm_code));
296
297     return v_counts;
298 end $$;
299
300 -- -----
301 -- 7. Admin visibility over every generated paper
302 --     Additive policies - existing teacher policies keep working.
303 -- -----
304 drop policy if exists papers_admin_select on public.papers;
305 create policy papers_admin_select on public.papers
306     for select using (public.is_admin());
307
308 drop policy if exists paper_questions_admin_select on public.paper_questions;
309 create policy paper_questions_admin_select on public.paper_questions
310     for select using (public.is_admin());
311
312 drop policy if exists exam_cycles_admin_select on public.exam_cycles;
313 create policy exam_cycles_admin_select on public.exam_cycles
314     for select using (public.is_admin());
315
316 -- Track who generated each paper (older rows stay null).
317 alter table public.papers
318     add column if not exists created_by uuid default auth.uid();
319
320 -- -----
321 -- 8. Grants - RPCs are callable by logged-in users; the role check
322 --     inside each function is what actually gates them.
323 -- -----
324 grant execute on function public.is_admin() to authenticated;
325 grant execute on function public.subject_delete_preview(uuid) to authenticated;
326 grant execute on function public.delete_subject(uuid, text) to authenticated;
327 revoke execute on function public.log_audit(text, text, uuid, text, jsonb) from authenticated,
    >> anon;

```

```

328
329 commit;
330
331 -- quick check
332 select action, count(*), max(at) as latest
333 from public.audit_logs group by action order by latest desc;

=====
FILE: question_images.sql (89 lines)
=====
1 -- =====
2 -- Image questions: storage bucket + access policies.
3 --
4 -- The object is named after the question it belongs to, so the
5 -- image can be located from the question row alone:
6 --     question-images/<question_id>.<ext>
7 -- The full object path is still persisted in questions.image_url,
8 -- because the extension varies and guessing it would mean probing
9 -- the bucket on every read.
10 --
11 -- The bucket is PRIVATE. Question images are part of the paper and
12 -- carry the same confidentiality as the question text, so they are
13 -- read through short-lived signed URLs, never a public URL.
14 --
15 -- Run in the Supabase SQL editor. Idempotent.
16 -- =====
17
18 begin;
19
20 -- 1. Bucket -----
21 insert into storage.buckets (id, name, public, file_size_limit,
22                             allowed_mime_types)
23 values ('question-images', 'question-images', false, 5242880,
24         array['image/png', 'image/jpeg', 'image/webp', 'image/gif'])
25 on conflict (id) do update
26     set public = false,
27         file_size_limit = excluded.file_size_limit,
28         allowed_mime_types = excluded.allowed_mime_types;
29
30 -- 2. Access -----
31 -- Signed URLs are issued against the caller's own read permission,
32 -- so authenticated users need select. Anonymous users get nothing.
33 drop policy if exists question_images_read on storage.objects;
34 create policy question_images_read on storage.objects
35     for select to authenticated
36     using (bucket_id = 'question-images');
37
38 drop policy if exists question_images_write on storage.objects;
39 create policy question_images_write on storage.objects
40     for insert to authenticated
41     with check (bucket_id = 'question-images');
42
43 drop policy if exists question_images_update on storage.objects;
44 create policy question_images_update on storage.objects
45     for update to authenticated
46     using (bucket_id = 'question-images')
47     with check (bucket_id = 'question-images');
48
49 drop policy if exists question_images_delete on storage.objects;
50 create policy question_images_delete on storage.objects
51     for delete to authenticated
52     using (bucket_id = 'question-images');
53
54 -- 3. Column -----
55 -- Already present in most installs; kept here so a fresh database
56 -- built from these files alone is complete.
57 alter table public.questions
58     add column if not exists image_url text;
59

```

```

60 comment on column public.questions.image_url is
61 'Object path inside the question-images bucket, named <question_id>.<ext>. '
62 'Null when the question has no image.';
63
64 -- 4. Housekeeping -----
65 -- Deleting a question leaves its image orphaned in the bucket.
66 -- Storage objects cannot be removed from SQL, so record the orphan
67 -- in the audit log and clear it out with a periodic job.
68 create or replace function public.audit_question_image_del()
69 returns trigger language plpgsql security definer set search_path = public as $$
70 begin
71     if old.image_url is not null then
72         perform public.log_audit(
73             'question.image_orphan', 'question', old.id,
74             format('Image %s orphaned by deletion of its question', old.image_url),
75             jsonb_build_object('image_url', old.image_url));
76     end if;
77     return old;
78 end $$;
79
80 drop trigger if exists trg_audit_question_image_del on public.questions;
81 create trigger trg_audit_question_image_del
82     before delete on public.questions
83     for each row execute function public.audit_question_image_del();
84
85 commit;
86
87 select count(*) filter (where image_url is not null) as with_image,
88        count(*) as total
89 from public.questions;

```

=====

FILE: seed_appmath2.sql (166 lines)

=====

```

1  -- =====
2  -- Seed: Applied Mathematics-II question bank (6 units)
3  -- Run in Supabase SQL editor of project dfkpsipqdkbrkxmtznh.
4  -- Prerequisites: schema (subjects, questions) already created.
5  -- Idempotent: re-running skips questions whose text already exists
6  -- for this subject. co_no is set = unit_no (C01..C06).
7  -- =====
8  begin;
9
10 with subj as (
11     insert into public.subjects (code, name, semester)
12     values ('CS/AI/ML201BSC03', 'Applied Mathematics-II', 2)
13     on conflict (code) do update
14         set name = excluded.name, semester = excluded.semester
15     returning id
16 )
17 insert into public.questions
18     (subject_id, unit_no, marks, difficulty, bt_level, co_no, text, is_active)
19 select s.id, v.unit_no, v.marks, v.difficulty, v.bt_level, v.unit_no, v.text, true
20 from subj s
21 cross join (values
22     -- ===== UNIT 1 : Logic, Sets, Monoids, Groups =====
23     (1, 1, 'easy', 1, 'Define Tautology.'),
24     (1, 1, 'easy', 2, 'Define Contradiction.'),
25     (1, 1, 'easy', 1, 'Define Predicate.'),
26     (1, 1, 'easy', 2, 'Define Universal Quantifier.'),
27     (1, 1, 'easy', 1, 'Define Existential Quantifier.'),
28     (1, 1, 'easy', 2, 'Define Monoid.'),
29     (1, 1, 'easy', 1, 'Define Group.'),
30     (1, 1, 'easy', 2, 'What is Identity element?'),
31     (1, 1, 'easy', 1, 'Define Partial Order.'),
32     (1, 1, 'easy', 2, 'Define Lattice.'),
33     (1, 4, 'medium', 3, 'Construct the truth table and determine whether [(P->Q)?(R->P)]->(R->Q) is
34         >> a tautology, contradiction, or contingency.'),
35     (1, 4, 'medium', 3, 'Given A={1,2,3,4} and R={(1,1),(2,2),(3,3),(4,4),(1,2),(2,3),(1,3)}, de

```

```

    >> termine whether R is reflexive, symmetric, antisymmetric and transitive. Hence deter
    >> mine whether it is a partial order. '),
35 (1, 4, 'medium', 3, 'Let A={1,2,3,6,12} under divisibility. Draw the Hasse diagram and deter
    >> mine greatest, least, maximal and minimal elements. '),
36 (1, 4, 'medium', 3, 'Let  $f(x)=3x \pmod 5$  from  $R \rightarrow R$ . Determine whether it is one-one, onto and biject
    >> tive. '),
37 (1, 4, 'medium', 3, 'Show whether  $(\mathbb{Z}_{10}, +)$  forms a group. Identify identity and inverses. '),
38 (1, 4, 'medium', 3, 'Verify using Venn diagram:  $A \cap (B \cap C) = (A \cap B) \cap (A \cap C)$ . '),
39 (1, 4, 'medium', 3, 'Determine whether  $aRb$  iff  $a \nmid b$  is divisible by 5 is an equivalence relat
    >> ion. '),
40 (1, 4, 'medium', 3, 'Construct truth table for  $[(P \rightarrow Q) \rightarrow R] \rightarrow [(P \rightarrow R) \rightarrow (Q \rightarrow R)]$ . '),
41 (1, 4, 'medium', 3, 'Determine all upper bounds, lower bounds, LUB and GLB for {2,4} under d
    >> ivisibility. '),
42 (1, 4, 'medium', 3, 'Show whether  $(\mathbb{N}, \times)$  is a monoid but not a group. '),
43
44 -- ===== UNIT 2 : Graph Theory & Combinatorics =====
45 (2, 1, 'easy', 1, 'Define Complete Graph. '),
46 (2, 1, 'easy', 2, 'Define Simple Graph. '),
47 (2, 1, 'easy', 1, 'Define Tree. '),
48 (2, 1, 'easy', 2, 'Define Matching. '),
49 (2, 1, 'easy', 1, 'Define Bipartite Graph. '),
50 (2, 1, 'easy', 2, 'Define Chromatic Number. '),
51 (2, 1, 'easy', 1, 'Define Connected Graph. '),
52 (2, 1, 'easy', 2, 'Define Recurrence Relation. '),
53 (2, 1, 'easy', 1, 'Define Generating Function. '),
54 (2, 1, 'easy', 2, 'Define Euler Graph. '),
55 (2, 4, 'medium', 3, 'Solve  $a(n)=3a(n-1)+2$ ,  $a(0)=4$ . Find first six terms. '),
56 (2, 4, 'medium', 3, 'Solve  $a(n)=2a(n-1)+n$ ,  $a(0)=1$ . '),
57 (2, 4, 'medium', 3, 'Find the coefficient of  $x^5$  in  $(2+3x)^8$ . '),
58 (2, 4, 'medium', 3, 'Find the generating function of 1,3,6,10,15,...'),
59 (2, 4, 'medium', 3, 'Find the recurrence relation satisfied by 1,4,9,16,25,...'),
60 (2, 4, 'medium', 3, 'A graph has 10 vertices and 15 edges. Determine whether it is connected
    >> and whether it can be a tree. '),
61 (2, 4, 'medium', 3, 'Determine the chromatic number of a given graph. '),
62 (2, 4, 'medium', 3, 'Determine whether a graph possesses an Euler path, Euler circuit and Ha
    >> miltonian cycle. '),
63 (2, 4, 'medium', 3, 'Find the number of spanning trees of the given graph. '),
64 (2, 4, 'medium', 3, 'Find the degree sequence and verify the Handshaking Lemma. '),
65
66 -- ===== UNIT 3 : Statistics =====
67 (3, 1, 'easy', 1, 'Define Mean. '),
68 (3, 1, 'easy', 2, 'Define Median. '),
69 (3, 1, 'easy', 1, 'Define Mode. '),
70 (3, 1, 'easy', 2, 'Define Standard Deviation. '),
71 (3, 1, 'easy', 1, 'Define Variance. '),
72 (3, 1, 'easy', 2, 'Define Correlation. '),
73 (3, 1, 'easy', 1, 'Define Covariance. '),
74 (3, 1, 'easy', 2, 'Define Frequency Distribution. '),
75 (3, 1, 'easy', 1, 'Define Class Interval. '),
76 (3, 1, 'easy', 2, 'Define Arithmetic Mean. '),
77 (3, 4, 'medium', 3, 'Calculate the arithmetic mean (Direct Method) of the grouped frequency
    >> distribution: Class Interval 10-20, 20-30, 30-40, 40-50, 50-60, 60-70; Frequency 5,
    >> 8, 15, 12, 7, 3. '),
78 (3, 4, 'medium', 3, 'Calculate the arithmetic mean using the Assumed Mean Method: Class Inte
    >> rval 0-10, 10-20, 20-30, 30-40, 40-50; Frequency 6, 9, 18, 10, 7. '),
79 (3, 4, 'medium', 3, 'Find the median of the grouped data: Class Interval 0-20, 20-40, 40-60,
    >> 60-80, 80-100; Frequency 5, 9, 14, 11, 6. '),
80 (3, 4, 'medium', 3, 'Calculate the mode of the grouped frequency distribution: Class Interva
    >> l 5-15, 15-25, 25-35, 35-45, 45-55; Frequency 4, 8, 15, 12, 6. '),
81 (3, 4, 'medium', 3, 'Calculate the standard deviation using the Direct Method: Class Interva
    >> l 20-30, 30-40, 40-50, 50-60, 60-70; Frequency 4, 7, 12, 10, 7. '),
82 (3, 4, 'medium', 3, 'For the discrete frequency distribution, calculate (i) Arithmetic Mean
    >> and (ii) Standard Deviation: Marks 15, 25, 35, 45, 55, 65; Frequency 5, 7, 10, 8, 6,
    >> 4. '),
83 (3, 4, 'medium', 3, 'Calculate the covariance between X and Y: X 2, 4, 6, 8, 10; Y 5, 9, 11,
    >> 15, 18. '),
84 (3, 4, 'medium', 3, 'Find Karl Pearson's coefficient of correlation for the paired observat
    >> ions: X 12, 18, 20, 25, 28, 30; Y 15, 20, 25, 30, 35, 38. '),
85 (3, 4, 'medium', 3, 'The arithmetic mean of the following distribution is 30. Find the missi

```

```

86     >> ng frequency x: Class Interval 10-20, 20-30, 30-40, 40-50; Frequency 6, x, 8, 5.)),
    (3, 4, 'hard', 4, 'The marks obtained by 50 students are: Class Interval 0-10, 10-20, 20-3
    >> 0, 30-40, 40-50, 50-60; Frequency 3, 7, 15, 12, 8, 5. Calculate (1) Arithmetic Mean,
    >> (2) Median, (3) Mode.'),
87
88 -- ===== UNIT 4 : Hypothesis Testing =====
89 (4, 1, 'easy', 1, 'Define Null Hypothesis.'),
90 (4, 1, 'easy', 2, 'Define Alternative Hypothesis.'),
91 (4, 1, 'easy', 1, 'Define Level of Significance.'),
92 (4, 1, 'easy', 2, 'Define t-distribution.'),
93 (4, 1, 'easy', 1, 'Define Chi-square distribution.'),
94 (4, 1, 'easy', 2, 'Define Confidence Interval.'),
95 (4, 1, 'easy', 1, 'When is z-test used?'),
96 (4, 1, 'easy', 2, 'When is t-test used?'),
97 (4, 1, 'easy', 1, 'Define Type-I Error.'),
98 (4, 1, 'easy', 2, 'Define Type-II Error.'),
99 (4, 4, 'medium', 3, 'Perform one-sample z-test (n=36, mean=53,  $\sigma=6$ ,  $H_0: \sigma=50$ ).'),
100 (4, 4, 'medium', 3, 'Perform one-sample t-test (n=16, mean=42, SD=5,  $H_0: \sigma=40$ ).'),
101 (4, 4, 'medium', 3, 'Construct a 99% confidence interval.'),
102 (4, 4, 'medium', 3, 'Perform Chi-square goodness-of-fit test.'),
103 (4, 4, 'medium', 3, 'Perform Chi-square test for independence.'),
104 (4, 4, 'medium', 3, 'Compare z-test and t-test numerically.'),
105 (4, 4, 'medium', 3, 'Perform a left-tailed hypothesis test.'),
106 (4, 4, 'medium', 3, 'Perform a right-tailed hypothesis test.'),
107 (4, 4, 'medium', 3, 'Determine acceptance and rejection regions.'),
108 (4, 4, 'medium', 3, 'Interpret the conclusion of a hypothesis test.'),
109
110 -- ===== UNIT 5 : Probability Distributions =====
111 (5, 1, 'easy', 1, 'Define Random Variable.'),
112 (5, 1, 'easy', 2, 'Define Binomial Distribution.'),
113 (5, 1, 'easy', 1, 'Define Poisson Distribution.'),
114 (5, 1, 'easy', 2, 'Define Normal Distribution.'),
115 (5, 1, 'easy', 1, 'Define Uniform Distribution.'),
116 (5, 1, 'easy', 2, 'Define Exponential Distribution.'),
117 (5, 1, 'easy', 1, 'Define Conditional Expectation.'),
118 (5, 1, 'easy', 2, 'Define Conditional Variance.'),
119 (5, 1, 'easy', 1, 'State the mean of the Binomial distribution.'),
120 (5, 1, 'easy', 2, 'State the variance of the Poisson distribution.'),
121 (5, 4, 'medium', 3, 'Binomial: 8% defective among 20 bulbs. Find exactly 3 and at most 4 def
    >> ectives.'),
122 (5, 4, 'medium', 3, 'Poisson: Average accidents=4/day. Find exactly 6 and at least 3 acciden
    >> ts.'),
123 (5, 4, 'medium', 3, 'Normal:  $N(120, 15^2)$ . Find probabilities above 135, below 90 and between
    >> 110 and 130.'),
124 (5, 4, 'medium', 3, 'A die is thrown 12 times. Find probability of at least two sixes.'),
125 (5, 4, 'medium', 3, 'A coin is tossed 10 times. Find probabilities of at least 7 heads and a
    >> t most 3 heads.'),
126 (5, 4, 'medium', 3, 'Uniform distribution: Find mean, variance and probability.'),
127 (5, 4, 'medium', 3, 'Exponential distribution: Find reliability after 10 hours and failure p
    >> robability before 8 hours.'),
128 (5, 4, 'medium', 3, 'Find conditional expectation from joint probabilities.'),
129 (5, 4, 'medium', 3, 'Find conditional variance.'),
130 (5, 4, 'medium', 3, 'Determine whether Poisson approximation to Binomial is appropriate.'),
131
132 -- ===== UNIT 6 : PMF, PDF, CDF, CLT =====
133 (6, 1, 'easy', 1, 'Define PMF.'),
134 (6, 1, 'easy', 2, 'Define PDF.'),
135 (6, 1, 'easy', 1, 'Define CDF.'),
136 (6, 1, 'easy', 2, 'Define Bernoulli Distribution.'),
137 (6, 1, 'easy', 1, 'Define Continuous Random Variable.'),
138 (6, 1, 'easy', 2, 'State one condition of a PMF.'),
139 (6, 1, 'easy', 1, 'State one condition of a PDF.'),
140 (6, 1, 'easy', 2, 'Define Standard Normal Distribution.'),
141 (6, 1, 'easy', 1, 'State the Central Limit Theorem (CLT).'),
142 (6, 1, 'easy', 2, 'Define Conditional PDF.'),
143 (6, 4, 'medium', 3, 'Given  $f(x)=kx^2$ ,  $0 < x < 2$ . Find k, CDF and  $P(1 < X < 1.5)$ .'),
144 (6, 4, 'medium', 3, 'Given exponential PDF  $f(x)=1/20 e^{-(x/20)}$ . Find  $P(X > 30)$  and  $P(10 < X < 25)$ .
    >> '),
145 (6, 4, 'medium', 3, 'Given PMF  $P(X=x)=k(x+1)$ ,  $x=0,1,2,3$ . Find k, mean and variance.'),

```

```
146     (6, 4, 'medium', 3, 'Construct Bernoulli PMF and derive mean and variance.'),
147     (6, 4, 'medium', 3, 'Find sampling distribution mean and standard error (?=75, ?=24, n=64).',
        >> ),
148     (6, 4, 'medium', 3, 'Using CLT, ?=100, ?=20, n=49. Find  $P(95 < X < 105)$ .'),
149     (6, 4, 'medium', 3, 'Verify whether a given function is a valid PDF.'),
150     (6, 4, 'medium', 3, 'Find CDF from a given PDF and calculate probabilities.'),
151     (6, 4, 'medium', 3, 'Solve probability using standard normal distribution.'),
152     (6, 4, 'medium', 3, 'Bernoulli distribution with  $p=0.3$ : Find PMF, CDF, mean, variance and  $P($ 
        >>  $X \leq 1)$ .'))
153 ) as v(unit_no, marks, difficulty, bt_level, text)
154 where not exists (
155     select 1 from public.questions q
156     where q.subject_id = s.id and q.text = v.text
157 );
158
159 commit;
160
161 -- quick check
162 select unit_no, marks, count(*)
163 from public.questions
164 where subject_id = (select id from public.subjects where code = 'CS/AI/ML201BSC03')
165 group by unit_no, marks
166 order by unit_no, marks;
```